24. Handle Eventual Consistency

Key Terms:

Term	Full Name	What it does
CAP Theorem	Consistency, Availability, Partition tolerance	Trade-off: can only guarantee 2 of 3
LWW	Last Write Wins	Conflict resolution using timestamps
CRDT	Conflict-free Replicated Data Type	Data structures that auto-merge
Vector Clock	Distributed timestamp	Tracks causality across nodes
Idempotency	Same result on retry	Operation can be safely repeated
Optimistic UI	Immediate feedback	Show change before server confirms

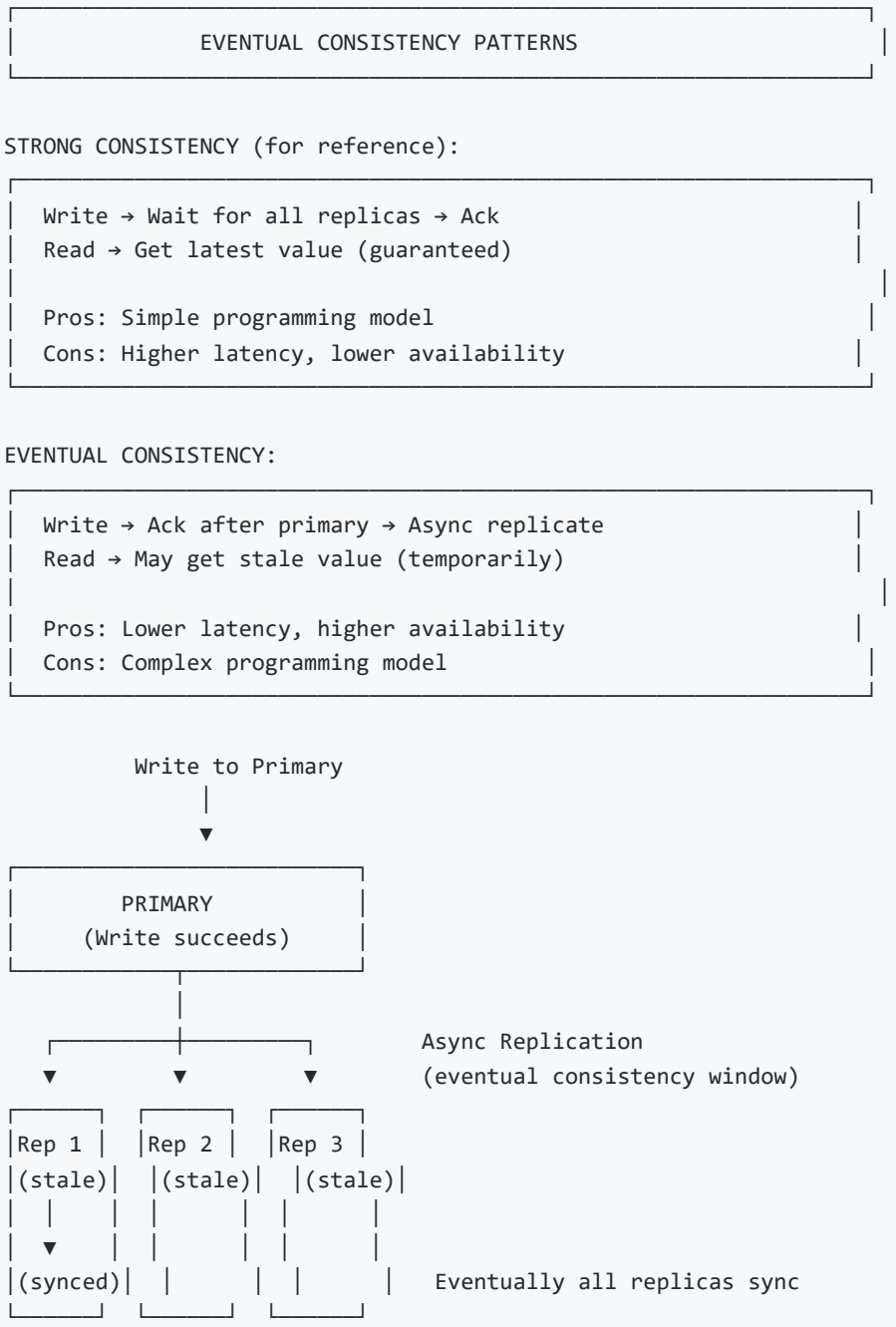
Problem Statement

Design strategies to handle eventual consistency in distributed databases while providing good user experience and data integrity.

Clarifying Questions to Ask

1. **Data type**: What data can tolerate eventual consistency?
2. **Delay tolerance**: How much delay is acceptable? (seconds? minutes?)
3. **Conflict resolution**: How to handle concurrent updates?
4. **Read patterns**: Do users read their own writes frequently?
5. **Cross-region**: Geographic distribution involved?
6. **Compensation**: Can incorrect operations be undone?
7. **Visibility**: Should users know about consistency delays?

High-Level Design



API Design

Consistency-Aware Read APIs

Method	Endpoint	Description
`GET`	`/api/v1/resources/{id}`	Read with eventual consistency (default)

`GET`	`/api/v1/resources/{id}?consistency=strong`	Read with strong consistency
`GET`	`/api/v1/resources/{id}?consistency=session`	Read-your-own-writes consistency
`GET`	`/api/v1/resources/{id}?consistency=bounded&staleness=5s`	Bounded staleness read

Write APIs (with consistency feedback)

```
POST /api/v1/resources
{
  "data": { ... },
  "consistency_level": "quorum"
}

Response: 201 Created
{
  "id": "res_123",
  "version": 5,
  "write_timestamp": 1672531200000,
  "replicated_to": ["region-1", "region-2"],
  "pending_regions": ["region-3"]
}
```

Conflict Resolution APIs

Method	Endpoint	Description
`GET`	`/api/v1/resources/{id}/versions`	List conflicting versions
`POST`	`/api/v1/resources/{id}/resolve`	Resolve conflict manually

Patterns for Handling Eventual Consistency

```
1. READ YOUR OWN WRITES
┌─────────────────────────────────────────────────────────────┐
│  Problem: User updates profile, refreshes, sees old data    │
│                                                             │
│  Solution A: Sticky sessions to primary                     │
│    - Route user's reads to primary after their writes       │
│    - Track: user_123 wrote at T, read from primary until T+5s  │
│                                                             │
│  Solution B: Version token                                  │
│    - Write returns version: v=42                            │
```

```
│    - Read includes: If-None-Match: v=42                          │
│    - If replica is behind, route to primary                      │
│                                                                   │
│  Solution C: Local cache + background refresh                    │
│    - Show optimistic update immediately                          │
│    - Background sync confirms or corrects                        │
└──────────────────────────────────────────────────────────────────┘

2. MONOTONIC READS
┌──────────────────────────────────────────────────────────────────┐
│  Problem: User sees v=5, refreshes, sees v=4 (went backward)     │
│                                                                   │
│  Solution: Session-consistent reads                              │
│    - Track highest version seen per user                         │
│    - Only serve reads from replicas at or above that version     │
│    - If no replica is current enough, wait or route to primary   │
└──────────────────────────────────────────────────────────────────┘

3. CAUSAL CONSISTENCY
┌──────────────────────────────────────────────────────────────────┐
│  Problem: User A posts, User B comments, User C sees comment     │
│           but not the original post (causality violated)         │
│                                                                   │
│  Solution: Causal ordering with vector clocks                    │
│    - Each write includes dependency list                         │
│    - Reads wait for dependencies to be visible                   │
│                                                                   │
│  Post: {id: 1, deps: []}                                         │
│  Comment: {id: 2, deps: [1]}                                     │
│  Read comment → ensure post 1 is visible first                   │
└──────────────────────────────────────────────────────────────────┘
```

Conflict Resolution Strategies

```
1. LAST WRITE WINS (LWW)
┌──────────────────────────────────────────────────────────────────┐
│  Each update has timestamp                                       │
│  Latest timestamp wins during merge                              │
│                                                                   │
│  Node A: {value: "A", ts: 100}                                   │
│  Node B: {value: "B", ts: 105}                                   │
│  Merge result: "B" wins                                          │
│                                                                   │
│  Pros: Simple, deterministic                                     │
│  Cons: Data loss (A's update silently lost)                      │
└──────────────────────────────────────────────────────────────────┘

2. MERGE FUNCTION (Application-specific)
┌──────────────────────────────────────────────────────────────────┐
│  Shopping cart example:                                          │
│  Node A: {items: ["apple", "banana"]}                            │
│  Node B: {items: ["apple", "cherry"]}                            │
│  Merge: {items: ["apple", "banana", "cherry"]} (union)           │
```

```
│                                                                 │
│  Counter example:                                               │
│  Node A: {count: +5}                                            │
│  Node B: {count: +3}                                            │
│  Merge: {count: +8} (sum of deltas)                             │
└─────────────────────────────────────────────────────────────────┘

3. OPERATIONAL TRANSFORMATION / CRDT

┌─────────────────────────────────────────────────────────────────┐
│  Design data structures that merge automatically                │
│                                                                 │
│  G-Counter (grow-only counter):                                 │
│  Node A: {A: 5, B: 0}                                           │
│  Node B: {A: 0, B: 3}                                           │
│  Merge: {A: 5, B: 3} → total = 8                                │
│                                                                 │
│  Pros: Always converges, no conflicts                           │
│  Cons: Limited data structure types                             │
└─────────────────────────────────────────────────────────────────┘
```

User Experience Patterns

```
OPTIMISTIC UI:

┌─────────────────────────────────────────────────────────────────┐
│  1. User clicks "Like"                                          │
│  2. Immediately show liked state (optimistic)                   │
│  3. Send request to server in background                        │
│  4. If fails → revert UI + show error                           │
│                                                                 │
│  User perceives instant response                                │
└─────────────────────────────────────────────────────────────────┘

LOADING STATES + CONFIRMATION:

┌─────────────────────────────────────────────────────────────────┐
│  1. User submits order                                          │
│  2. Show "Processing..." state                                  │
│  3. Poll or WebSocket for confirmation                          │
│  4. Show "Order confirmed" when synced                          │
└─────────────────────────────────────────────────────────────────┘

IDEMPOTENCY KEYS:

┌─────────────────────────────────────────────────────────────────┐
│  Client generates unique key per operation:                     │
│    POST /orders                                                 │
│    Idempotency-Key: order-abc123                                │
│                                                                 │
│  Server deduplicates:                                           │
│    If seen → return cached response                             │
│    If new → process and store response                          │
│                                                                 │
│  Safe to retry on timeout/failure                               │
└─────────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Option A	Option B	Recommendation
Conflict resolution	LWW	Custom merge	**Domain-specific**
User writes	Read-after-write	Optimistic UI	**Optimistic** - better UX
Retries	Auto-retry	Idempotency	**Idempotency keys**
Visibility	Hide delays	Show status	**Show** - transparency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Raft Consensus	Leader-based replication for strong consistency when needed	Tech Guide

How to Explain It Well

> **Opening**: "Eventual consistency trades immediate consistency for availability. I'll design patterns for good UX despite replication delays."

> **Key talking points**:
>
> - "Read-your-own-writes: route reads to primary after user's writes"
> - "Optimistic UI: show changes immediately, reconcile in background"
> - "Idempotency keys: safe retries without duplicate operations"
> - "CRDTs: design data structures that merge automatically"
> - "Conflict resolution depends on domain: LWW for profiles, merge for carts"
